

Redes de Datos II

Facultad de Informática — UNLP

Taciano Pacchialat

2026

Resumen

Este documento es una guía estructurada para el repaso de los contenidos fundamentales de la materia. Se enfoca en la claridad visual y la jerarquización de conceptos mediante el uso de colores y bloques resaltados.

Índice

1. Address Resolution Protocol (ARP)	4
1.1. Funcionamiento de ARP	4
2. Protocolo IP	6
2.1. Diseño y Servicios	6
2.2. Estructura del Datagrama IP	7
2.3. Direccionamiento IP	8
2.3.1. Subnetting	8
2.4. VLSM (Variable Length Subnet Mask)	9
2.4.1. Supernetting (Sumarización)	9
2.4.2. Cuadro Comparativo	10
2.5. Routing	10
2.6. Fragmentación	11
3. Internet Control Message Protocol (ICMP)	12
3.1. Mensajes de Error (Error-reporting messages)	12
3.2. Mensajes de Consulta (Query messages)	12
4. Dynamic Host Configuration Protocol - DHCP	13
4.1. Funcionamiento de DHCP	13
5. Protocolo IPv6	15
5.1. Cambios con respecto a IPv4	15
5.2. Direcciones IPv6	15

6. ICMPv6 con NDP	16
6.1. Funciones Principales	16
6.2. Paquetes de NDP (Mensajes ICMPv6)	16
6.2.1. Mensajes entre Router y Host	16
6.2.2. Mensajes entre Vecinos (Resolución ARP en IPv6)	16
6.2.3. Redirección	17
7. Network Address Translation	18
7.1. NAT Básico	18
7.2. Network Address Port Translation (NAPT)	18
7.3. Port Forwarding	18
8. User Datagram Protocol	19
8.1. Protocolos de Transporte	19
8.1.1. Puertos	19
8.2. Sockets	19
8.3. Características de UDP	20
9. Transmission Control Protocol (TCP)	21
9.1. Segmento de TCP	21
9.2. Three-Way Handshake	22
9.2.1. Reset de Conexiones	22
9.2.2. Fin de Conexión	23
9.2.3. Estados de TCP	23
9.3. Intercambio de Datos	24
9.4. Control de Flujo	25
9.4.1. Timers	25
9.5. Control de Congestión	26
9.5.1. Modelo end-to-end	26
9.5.2. Fast Retransmit, Fast Recovery	27
10. QUIC	29
10.1. Paquetes y Frames	29
10.2. Streams	29
10.3. Conexiones	30
10.3.1. Handshake	30
10.3.2. Cierre de Conexiones	31
10.4. Control de Flujo	32
10.5. Transport Layer Security (TLS) 1.3	32

11.Domain Name System	34
11.1.Registros de Recursos	34
11.2.Servidores DNS	35
11.3.Proceso de Consulta	35
11.4.Flags DNS	36
12.File Transfer Protocol - FTP	37
13.Correo Electrónico	38
13.1.Simple Mail Transfer Protocol - SMTP	38
13.1.1.Multipurpose Internet Mail Extensions - MIME	39
13.2.Post Office Protocol 3 - POP3	39
13.3.IMAP	39
14.Hypertext Transfer Protocol - HTTP/1.1	40
14.1.Mensajes y Métodos	40
14.1.1.Algunos Métodos	40
14.2.Funcionamiento	41
14.3.HTTPS	42
14.4.Cookies	43
14.5.Caching	43
15.HTTP/2	45
15.1.Frames	45
15.2.Streams	46
15.3.Control de Flujo	47
15.4.Server Push	47
15.5.Head Compression	47

1 Address Resolution Protocol (ARP)

ARP es un protocolo de capa 2 (capa de enlace), que trabaja con Ethernet y otros protocolos de capa 2 para ayudar a IP. Es un protocolo **helper** de IP. Su función principal es mapear direcciones lógicas a físicas (mapear dirección IP a dirección MAC). Utiliza auto-aprendizaje, no requiere de configuración (es dinámico). Es encapsulado dentro de una trama Ethernet.

Recordar

ARP funciona dentro de un dominio de broadcast, no permite determinar la dirección MAC de un dispositivo en otra red.

Cada nodo tiene una tabla ARP en memoria que contiene la relación entre una IP y una MAC.

1.1 Funcionamiento de ARP

Supongamos que PC-A y PC-B están en un mismo dominio de broadcast. Si A le quiere enviar un mensaje a B, debe primero conocer su dirección MAC. Por la IP A sabe que B está en su misma red, entonces envía un **ARP Request** a la MAC de broadcast (ff:ff:....:ff) con la IP a la que debe enviarle el mensaje.

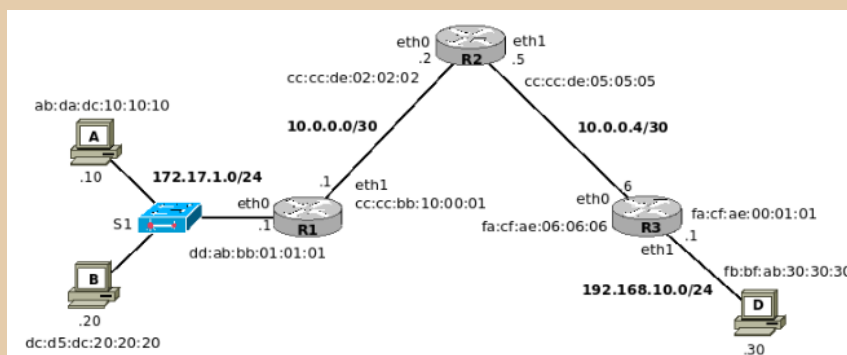


Figura 1: Topología del ejemplo

El nodo que tenga la IP destino asignada, procesará y responderá al ARP Request con un **ARP Reply** de tipo unicast. Una vez respondido, A ya puede enviarle mensajes a B por tener su dirección física.

Ahora supongamos que PC-A le quiere enviar un mensaje a PC-D, que está en **otra red**. Para llegar a D, A debe encontrar la MAC de su *default gateway*. El procedimiento es el mismo que para el caso anterior: ARP Request a la dirección de broadcast y el router de gateway responde con un ARP Reply.

Ahora, A envía el mensaje **Echo Request** con la interfaz del router como MAC destino. Cuando llega el mensaje, el router rutea el paquete y lo pasa a la capa inferior para for-

wardearlo por la interface que corresponda. El paquete puede ser enviado a otro router o al destino final, y si el router no conoce la MAC destino debe volver a ejecutar ARP para cambiar los campos de la trama Ethernet.

2 Protocolo IP

La **capa de enlace** (capa 2) se encarga de proveer conectividad entre nodos adyacentes o vecinos, siendo responsable de transmitir una trama sobre un enlace en particular. Los mensajes entre origen y destino pueden transitar por distintas capas de enlace. La **Capa de Red** se encuentra por encima de la capa de enlace.

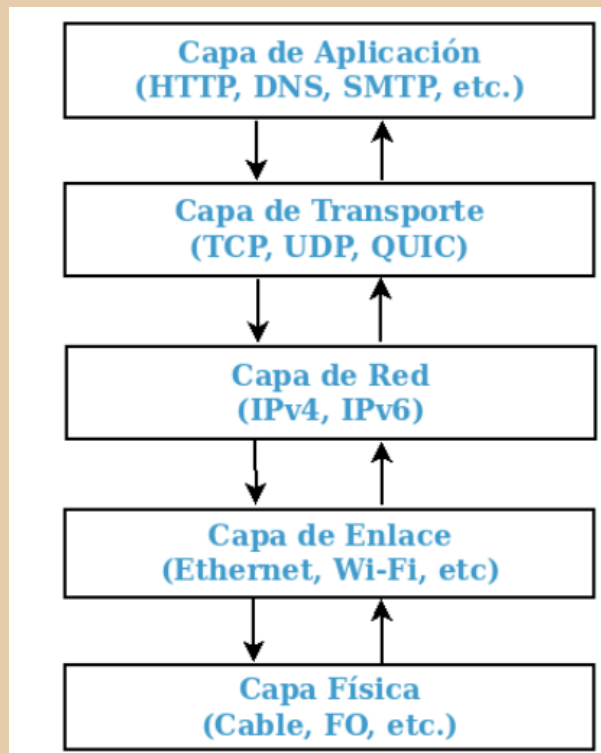


Figura 2: Modelo TCP/IP

2.1 Diseño y Servicios

La capa de red fué diseñada para que cumpla con los siguientes requerimientos:

- Independiente de la tecnología de subred.
- Aislada de la cantidad, tipo y topologías de las **subredes**.
- Direccionamiento uniforme.

Como servicios, debe poder transportar mensajes desde un nodo emisor a receptor (inclusive si son redes distintas) y determinar el camino que seguirá un mensaje entre dos nodos. Al no ser orientado a conexión, no brinda ni control de errores, ni secuenciamiento ni confiabilidad: es un servicio de **Best-Effort**.

Protocolo Best-Effort

Hay independencia en cuanto al envío a través de la red, haciendo posible que los paquetes puedan llegar desordenados o seguir caminos diferentes. Tampoco hay control de errores.

Tiene dos funcionalidades principales:

- Direccionamiento
- Ruteo

También se encarga de multiplexar o demultiplexar protocolos superiores, y puede fragmentar paquetes.

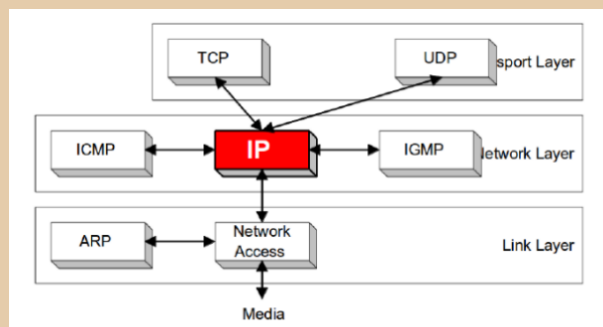


Figura 3: Modelo TCP/IP

2.2 Estructura del Datagrama IP

El **Datagrama IP** tiene longitud variable. Está compuesto por dos partes: header y payload. El **Header** tiene una longitud de entre 20 y 60 bytes.

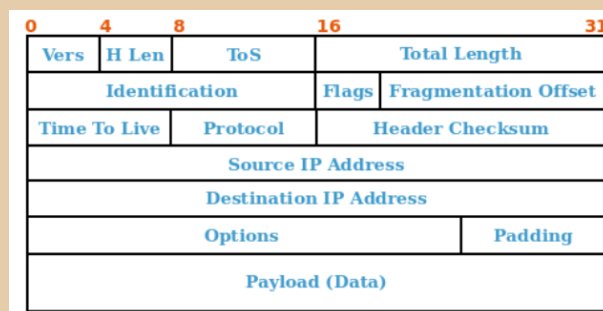


Figura 4: Datagrama IP

Los campos más relevantes son:

- **Header Length:** longitud del header, en palabras de 32 bits.
- **Type of Service (TOS):** confiabilidad, prioridad, retardo y throughput.

- **Identification:** número único para segmentos del mismo datagrama.
- **Flags:**
 - X = reservado.
 - D = Don't Fragment. Si es 0 puede fragmentarse, si es 1 no.
 - M = More Fragments. Si es 0 es el último fragmento, si es 1 no.
- **Fragment Offset:** el desplazamiento del segmento dentro del datagrama original. Está dado en múltiplos de 8 bytes.
- **Time To Live (TTL):** número que se decrementa cada vez que llega a un router, y cuando llega a 0 se descarta. Es utilizado para evitar loops.
- **Protocol:** protocolo contenido en el campo de datos (UDP, TCP, ICMP, ...).

2.3 Direccionamiento IP

Las direcciones IP identifican unívocamente una interfaz de red. Los routers pueden tener más de una dirección IP, y a diferencia de las direcciones MAC, estas direcciones son lógicas.

Hay distintos tipos de direcciones:

- **Unicast:** destinada a un host/interface en particular.
- **Broadcast:** destinada a todos los hosts de una red.
- **Multicast:** destinada a un grupo de hosts de una o varias redes.

Además de estas direcciones, también está la de *Loopback* (127.0.0.1) y la utilizada cuando aún no se le asignó una red al dispositivo (0.0.0.0).

2.3.1 Subnetting

El subnetting es el proceso de dividir una red física o lógica de gran tamaño en subredes más pequeñas. Esto se logra alterando la máscara de red predeterminada, "pidiendo prestados" bits de la porción destinada a los hosts para asignarlos a la porción de red.

- **Objetivo:** Reducir el tamaño de los dominios de broadcast, mejorar la seguridad, facilitar la administración y evitar el desperdicio de direcciones IP dentro de una organización.
- **Mecanismo:** Se desplaza el límite de la máscara de red hacia la derecha (aumenta el prefijo).

- **Cálculo de Subredes y Hosts:** Si se toman n bits prestados de la porción de host, se pueden crear 2^n subredes. En cada subred, si quedan h bits para los hosts, habrá $2^h - 2$ direcciones utilizables (se restan 2 por las direcciones de red y de broadcast).

2.4 VLSM (Variable Length Subnet Mask)

El subnetting tradicional (FLSM) divide una red en subredes del mismo tamaño, lo que a menudo sigue generando desperdicio. **VLSM** mejora esto permitiendo aplicar diferentes máscaras de subred dentro del mismo bloque de direcciones mayor. Al hacer subnetting de una subred previamente dividida, los bloques se ajustan estrictamente a la cantidad de hosts necesarios para cada departamento o enlace.

Ejemplo de Subnetting

Dada la red 192,168,10,0/24, si necesitamos una red para 80 hosts (requiere 7 bits, $2^7 - 2 = 126$), utilizamos un prefijo /25. La dirección sería 192,168,10,0/25. El segmento sobrante, 192,168,10,128/25, se puede seguir dividiendo con prefijos mayores (como /26, /27) para áreas que requieran menos hosts.

2.4.1 Supernetting (Sumarización)

El supernetting o **CIDR (Classless Inter-Domain Routing)**, es el proceso conceptualmente opuesto al subnetting. Consiste en combinar o agrupar múltiples redes contiguas más pequeñas en una única red o ruta más grande.

- **Objetivo:** Reducir drásticamente el tamaño de las tablas de ruteo en los routers de núcleo (core routers) de Internet y optimizar el procesamiento de rutas.
- **Mecanismo:** Se desplaza el límite de la máscara de red hacia la izquierda (disminuye el tamaño del prefijo), agrupando los bits de red compartidos.
- **Requisitos para la Sumarización:** Para agrupar redes, estas deben ser contiguas, el número de redes a agrupar debe ser potencia de 2, y las redes deben compartir exactamente los mismos bits de red de orden superior.

Ahora un **ejemplo**. Supongamos que un router tiene en su tabla las siguientes cuatro rutas hacia subredes contiguas:

- 192,168,0,0/24 (En binario el tercer octeto es: 00000000)
- 192,168,1,0/24 (En binario el tercer octeto es: 00000001)
- 192,168,2,0/24 (En binario el tercer octeto es: 00000010)

- 192,168,3,0/24 (En binario el tercer octeto es: 00000011)

Dado que las cuatro redes comparten los primeros 22 bits (192,168,000000XX), se pueden agrupar en una única entrada en la tabla de enrutamiento reduciendo la máscara a /22. La ruta resumida resultante será:

192,168,0,0/22

2.4.2 Cuadro Comparativo

Característica	Subnetting	Supernetting (CIDR)
Definición	Divide una red en redes más pequeñas.	Agrupar redes pequeñas en una más grande.
Movimiento de Máscara	Hacia la derecha (aumenta el prefijo, ej. /24 → /26).	Hacia la izquierda (reduce el prefijo, ej. /24 → /22).
Problema que Resuelve	Agotamiento interno de IPs y dominios de broadcast grandes.	Crecimiento desmedido de las tablas de ruteo globales.
Bits alterados	Toma bits de <i>Host</i> para hacer <i>Red</i> .	Toma bits de <i>Red</i> para hacer <i>Host</i> .

2.5 Routing

Acción donde se selecciona el próximo salto de un paquete, y su interfaz de salida. Los routers y hosts pueden hacer routing. Es una acción de control, cuyo motor son los protocolos de enrutamiento. Las tablas de ruteo están compuestas por:

- Red Destino
- Máscara de Red (o Prefijo)
- Next Hop (Próximo Salto)
- Interface de Salida
- Métrica

Recordar

Solo puede haber un default gateway por tabla de ruteo.

A diferencia del routing, el **Forwarding** consiste en pasar un mensaje desde una interfaz de entrada hacia una de salida, por donde envía los mensajes de los protocolos enrutados.

2.6 Fragmentación

Maximum Transfer Unit (MTU)

Es el tamaño máximo de paquete permitido en un enlace.

Un datagrama puede pasar por varias capas de enlaces con distintos MTUs, y los datagramas con tamaño mayor al MTU deben ser fragmentados con tamaño **múltiplo de 8 bytes** (excepto el último). Cada fragmento pasa a ser un nuevo paquete, donde el header se copia del original y luego se modifican los campos necesarios.

El **Path MTU** permite saber el mínimo MTU entre origen y destino.

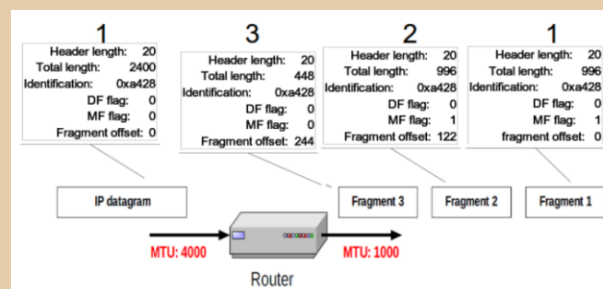


Figura 5: Fragmentación en IP

Los datagramas se reconstruyen en el destino final una vez que llegan todos los fragmentos, que pueden tomar distintos caminos. Estos fragmentos pueden a su vez fragmentarse de nuevo. En caso de que no lleguen **todos** los fragmentos, el paquete entero se descarta.

3 Internet Control Message Protocol (ICMP)

Protocolo helper de IP completamente **informativo**, es decir, no toma decisiones sino que permite reportar errores o consultar información. Tampoco agrega confiabilidad, sino que **brinda feedback**. Se encapsula en IP, pero no es un protocolo de transporte.

Cuando un dispositivo descarta un datagrama IP, genera un mensaje ICMP que incluye el encabezado (header) del datagrama original que produjo el problema y al menos los primeros 8 bytes del campo de datos de dicho datagrama.

Los mensajes ICMP son los siguientes.

3.1 Mensajes de Error (Error-reporting messages)

Estos mensajes reportan errores que un dispositivo puede encontrar cuando procesa un paquete IP.

- **Destination Unreachable (Tipo 3):** Indica que el paquete no pudo llegar a su destino.
 - *Code 0:* Net Unreachable (Red inalcanzable).
 - *Code 1:* Host Unreachable (Host inalcanzable).
 - *Code 3:* Port Unreachable (Puerto inalcanzable, muy común al fallar una conexión UDP).
- **Time Exceeded (Tipo 11):** Se genera cuando expira un temporizador relacionado al datagrama.
 - *Code 0:* In Transit. Se venció el campo TTL (Time to Live) durante el tránsito por los routers.
 - *Code 1:* Reassembly Timeout. Se descartó un datagrama fragmentado porque se agotó el tiempo de espera antes de que llegaran todos sus fragmentos para el reensamblado.

3.2 Mensajes de Consulta (Query messages)

Estos mensajes ocurren usualmente de a pares (una solicitud y su correspondiente respuesta) y permiten obtener información específica del estado de un dispositivo en la red.

- **Echo Request / Echo Reply (Tipos 8 y 0):** Son los mensajes utilizados por la clásica utilidad de diagnóstico `ping` para comprobar la conectividad bidireccional entre nodos.
- **Timestamp Request / Timestamp Reply (Tipos 13 y 14):** Utilizados para determinar el tiempo de tránsito a través de la red o para sincronizar relojes de los dispositivos.

Cuando un mensaje ICMP es descartado, **no se genera otro** para avisar que se descartó (evito bucles). Solo avisamos el descarte de un fragmento si es el **primero**, y tampoco avisamos si se descarta un mensaje con destino multicast.

4 Dynamic Host Configuration Protocol - DHCP

Permite una asignación automática de dirección al nodo, con parámetros de configuración que recibe desde un servidor. Se ejecuta **encapsulado en UDP**; los clientes escuchan en el puerto 68 y los servidores en el 67.

DHCP contiene dos componentes: un protocolo para enviar parámetros de configuración desde el servidor DHCP a los hosts, y otro para alocar las direcciones de red a los hosts. Los mecanismos para alocar una dirección IP son tres:

- **Automatic Allocation:** asigna una dirección a un cliente permanentemente.
- **Dynamic Allocation:** asigna una dirección por tiempo limitado.
- **Manual Allocation:** la dirección se asigna por el administrador de red y DHCP solo se la transporta al cliente.

Este protocolo utiliza un **pool de direcciones**, que representa el rango de direcciones que el servidor puede asignar. Cuando se asigna una dirección se hace con un **lease**, que representa el tiempo por el que un host puede utilizar esa dirección. El host puede pedir una renovación del lease con otra solicitud.

Cuando el cliente solicita una IP también pide la máscara de red, el default gateway y un servidor DNS. También puede pedir servidores SMTP, POP, HTTP o NTP.

4.1 Funcionamiento de DHCP

DHCP funciona con un procedimiento llamado **DORA**: Discover, Offer, Request, Acknowledge.

1. El cliente envía un broadcast **DHCP Discovery** usando como origen la IP 0.0.0.0, indicando que no puede recibir unicasts (no tiene dirección).
2. El servidor (o servidores) responde con un **DHCP Offer** con una IP de su *pool*.
3. Si el cliente acepta esa IP, responde con un broadcast **DHCP Request**. Es broadcast para que los otros servidores sepan que su *Offer* fué rechazado.
4. El cliente comprueba que esa dirección ofrecida no esté en uso mediante ARP.
5. El servidor envía un **DHCP Ack** con parámetros de configuración y termina el proceso

Cuando tenemos varias redes, se suele usar un único servidor DHCP centralizado. Para que esto funcione existen los **DHCP Relays**, routers que reciben los mensajes broadcast de los clientes y se los reenvían al DHCP Server como unicast. También realiza el proceso inverso.

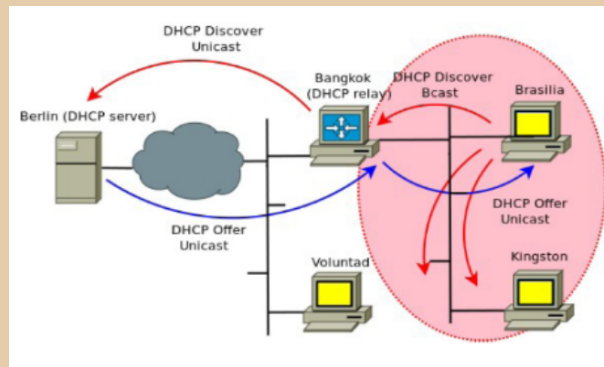


Figura 6: Ejemplo de DHCP Relay

5 Protocolo IPv6

A medida que avanzaba el tiempo, las direcciones IPv4 escaseaban, habían tablas de ruteo muy grandes y mucha congestión entre los routers por exceso de procesamiento. Por esto se introduce un protocolo nuevo: IPv6.

5.1 Cambios con respecto a IPv4

Los aspectos modificados son los siguientes:

- Direcciones de 128 bits.
- No hay más fragmentación.
- No hay más header checksum.
- El header es de tamaño fijo, sin campo de opciones.
- Se extiende el protocolo con cabeceras de extensión.

Ver.	TrafficClass	Flow Label	
Payload Length		Next Header	Hop Limit
128 bit Source Address			
128 bit Destination Address			

Figura 7: Header de IPv6

5.2 Direcciones IPv6

También se modifican los tipos de direcciones: ya no hay direcciones de broadcast, sino que hay unicast, anycast y multicast, que tienen formato FF00::/8.

La notación cambia. Los ceros al inicio de cada grupo de 2 bytes delimitados por : se pueden obviar, como también los ceros contiguos con ::, aunque este se puede utilizar una sola vez por dirección.

Tampoco existe la máscara de red, ya que el prefijo es /64 para las subredes y los link-local. La dirección de loopback ahora es ::1/128.

6 ICMPv6 con NDP

El ICMPv6 es una parte fundamental del stack IPv6. Se utiliza como protocolo de control, información y reporte de errores. Además, a diferencia de ICMP, incorpora el **Neighbor Discovery Protocol (NDP)**, reemplazo de ARP.

NDP es la funcionalidad más importante de ICMPv6. Su objetivo es mapear direcciones lógicas IPv6 a direcciones físicas. Al igual que ARP, trabaja dinámicamente con auto-aprendizaje sin la necesidad de una configuración previa.

A diferencia de ARP en IPv4, que utilizaba mensajes de *broadcast* (generando interrupciones a todos los nodos de la red), NDP opera sobre **ICMPv6** y utiliza direcciones de **multicast** específicas, lo que lo hace mucho más eficiente.

6.1 Funciones Principales

NDP proporciona varios servicios críticos para el funcionamiento de una red local IPv6:

- **Resolución de Direcciones (Address Resolution):** Sustituye a ARP, mapeando una dirección IPv6 conocida a una dirección física (MAC).
- **Descubrimiento de Routers (Router Discovery):** Permite a los hosts encontrar routers conectados en su mismo enlace local.
- **Autoconfiguración (SLAAC):** Permite a los hosts aprender los prefijos de red para generar automáticamente sus propias direcciones IPv6.
- **Detección de Direcciones Duplicadas (DAD):** Mecanismo para asegurar que una dirección IPv6 no esté siendo usada por otro nodo en la misma red antes de asignársela.

6.2 Paquetes de NDP (Mensajes ICMPv6)

NDP utiliza 5 tipos de mensajes ICMPv6 para llevar a cabo sus tareas. Se dividen principalmente en dos grupos: los relacionados con routers y los relacionados con vecinos (hosts).

6.2.1 Mensajes entre Router y Host

- **Router Solicitation (RS - Tipo 133):** Un host envía este mensaje (usualmente en multicast) cuando se conecta a la red para pedir a cualquier router presente que se anuncie inmediatamente, sin esperar al anuncio periódico.
- **Router Advertisement (RA - Tipo 134):** Los routers envían este mensaje de forma periódica o en respuesta a un RS. Contiene información vital para los hosts, como prefijos de red (para SLAAC), dirección del router por defecto, valor del MTU y límites de saltos.

6.2.2 Mensajes entre Vecinos (Resolución ARP en IPv6)

- **Neighbor Solicitation (NS - Tipo 135):** Es el equivalente al *ARP Request* de IPv4. Un nodo lo envía para consultar la dirección MAC (Link-Layer Address) de una dirección IPv6 objetivo. También se usa para el proceso DAD (verificar si una IP ya existe).
- **Neighbor Advertisement (NA - Tipo 136):** Es el equivalente al *ARP Reply*. Un nodo responde con este mensaje indicando su dirección MAC tras recibir un NS.

6.2.3 Redirección

- **Redirect (Tipo 137):** Un router utiliza este mensaje para informarle a un host que existe un camino (o un router de primer salto) mucho mejor para alcanzar un destino específico.

7 Network Address Translation

Utilizado para **traducir direcciones** desde un espacio privado a un espacio público. NAT es importante debido a que permite que un mismo conjunto de direcciones sea utilizado en distintas partes de internet como redes privadas. Los bloques de direcciones privadas son:

- 10.0.0.0/8
- 172.16.0.0/12
- 192.168.0.0/16

Hay routers encargados de hacer la translación, generalmente los routers de borde. Estos mantienen tablas de translación, y todo el tráfico de una sesión tiene que pasar por el mismo router NAT.

7.1 NAT Básico

El NAT básico solo reescribe y reemplaza direcciones IP, es decir, se mapea una IP privada a una pública. Si lo hacemos estáticamente, requerimos una dirección pública por dirección privada. Si es dinámico, necesitamos de un timer para cada entrada.

7.2 Network Address Port Translation (NAPT)

Debido a que NAT no es recomendado para un pool pequeño de direcciones, tenemos la opción de usar NAPT. NAPT es **One-to-Many**: utiliza los puertos de los protocolos de capa superior (TCP/UDP) para resolver el mapping.

A diferencia de NAT, ahora mantenemos en la tabla de traducción el protocolo, la dirección y el puerto origen y destino. El puerto origen siempre se intenta conservar, pero si ya está ocupado por otra traducción debemos reemplazarlo por otro que esté disponible.

Tenemos dos alternativas. Una es utilizando un **pool de direcciones**, donde configuramos una determinada cantidad de direcciones y hacemos *Port Address Translation* sobre estas direcciones del pool.

La otra alternativa es utilizar **Overloading**, donde para salir a internet los dispositivos de una red pequeña comparten una única dirección pública, y cada uno va tomando puertos a medida que los necesita.

7.3 Port Forwarding

Las técnicas que vimos anteriormente sólo permiten enviar tráfico de conexiones generadas internamente; no permiten el acceso desde afuera hacia adentro.

El *Port Forwarding* permite acceder a servicios ubicados en una red privada desde afuera de la misma. Sólo se implementa con NAPT, haciendo un mapeo reverso estático de puertos.

Resumen

NAT solo reemplaza una IP por otra, mientras que **NAPT** mapea IPs a puertos. **Port Forwarding** se usa para acceder desde afuera hacia adentro de una red privada.

8 User Datagram Protocol

8.1 Protocolos de Transporte

Protocolos de Transporte

Proveen **comunicación lógica entre procesos de aplicación** ejecutándose en distintos hosts. Se implementan sólo en los **extremos**, no en los dispositivos intermedios.

Reciben los datos de la capa de aplicación, los divide en pedazos y les agrega una nueva cabecera. Proveen multiplexación y demultiplexación de aplicaciones.

Actúan como una interface entre la **aplicación** y el **sistema operativo**.

8.1.1 Puertos

Los puertos son una abstracción identificada con un número, cuya implementación depende de cada sistema operativo.

Identifican unívocamente la **aplicación** transmisora y receptora.

Hay tres tipos de puertos:

- **Well-known:** 0 ... 1023.
- **Registered:** 1024 ... 49151.
- **Ephemeral:** 49152 ... 65535.

8.2 Sockets

Definición

Los sockets son la interface entre la capa de aplicación y la capa de transporte, mediante la cual los procesos se comunican escribiendo y leyendo.

La **dirección de un socket** está compuesta por:

- Protocolo
- Proceso local
- Dirección local

Y la **sesión** es: [protocolo, proceso local, dirección local, proceso remoto, dirección remota]

Recordar *En resumen, la capa de transporte aporta comunicación lógica entre procesos mediante puertos y sockets.*

8.3 Características de UDP

UDP

UDP es un protocolo de transporte **Best-Effort**, sin conexión: no hay ni handshake y los datagramas son independientes. No provee mecanismos de control de errores, flujo o congestión.

La estructura del datagrama es la siguiente.



Figura 8: Datagrama de UDP

El tamaño de la **cabecera** siempre es de 8 Bytes, y el checksum es opcional.

UDP también tiene una **pseudocabecera** que se añade al principio del datagrama para el cálculo del checksum, pero **no se envía**. Sirve para comprobar que IP no se haya equivocado en la entrega del datagrama.

El datagrama de UDP se **encapsula** en un paquete IP.

9 Transmission Control Protocol (TCP)

TCP brinda servicios a las aplicaciones, pero es orientado a conexión. Esto agrega:

- Confiabilidad
- Ordenado
- Control de flujo
- Control de congestión
- Byte stream

Las aplicaciones envían streams de bytes. TCP hace la **packetization**: convierte el stream en un conjunto de mensajes que IP puede transportar.

Después decide el tamaño de cada **chunk** para que no tenga que ser fragmentado. Estos chunks se llaman **segmentos**. Los segmentos contienen información sobre la posición en el stream, además del proceso origen del nodo emisor y el proceso destino en el nodo destino.

9.1 Segmento de TCP

El header del segmento tiene esta forma:

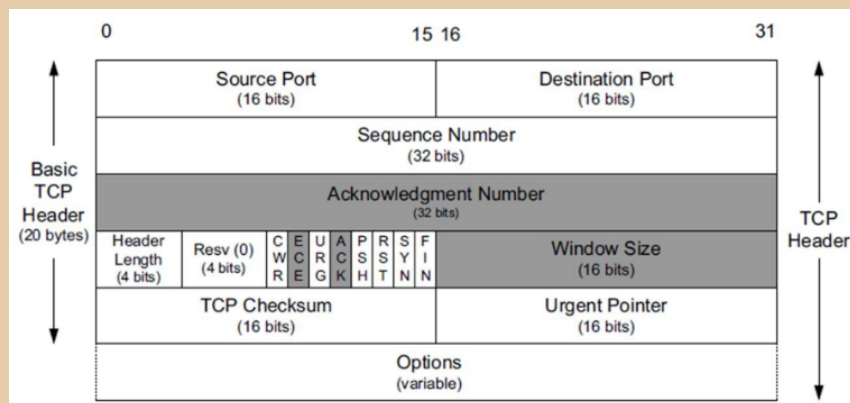


Figura 9: Segmento TCP

Y sus campos son los siguientes:

Cuadro 1: Campos de la cabecera TCP

Campo	Tamaño	Descripción
Source Port	16 bits	Indica el número de puerto de la aplicación en el nodo que envía el segmento.
Destination Port	16 bits	Indica el número de puerto de la aplicación que debe recibir el segmento.
Sequence Number	32 bits	Indica el número asignado al primer byte de datos contenidos en el segmento.
Acknowledgment Number	32 bits	Indica el siguiente número de byte que el emisor del segmento espera recibir del otro extremo.
Header Length	4 bits	Indica el número de palabras de 4 bytes de la cabecera TCP (entre 20 y 60 bytes).
Control Bits (Flags)	Variable	También conocidos como <i>flags</i> ; indican la finalidad del segmento.
Window Size	16 bits	Indica la cantidad de bytes, a partir del número de reconocimiento en el segmento, que el emisor puede recibir.
Checksum	16 bits	Su cálculo es obligatorio y se realiza sobre todo el segmento más una pseudocabecera.
Urgent Pointer	16 bits	Válido si el flag <i>U</i> está seteado. Define el valor a sumar al número de secuencia que indica dónde finalizan los datos urgentes.
Options	Hasta 40 bytes	Información opcional.

9.2 Three-Way Handshake

Se intercambian **tres segmentos**, donde se indican puertos origen y destino.

9.2.1 Reset de Conexiones

Se indica con el **Flag R** en un segmento, que provoca el cierre inmediato de la conexión. Este reset se puede hacer el cualquier momento, y TCP también lo puede hacer ante determinados eventos.

El reset es enviado cuando se recibe un segmento que **no corresponde** a la conexión referenciada. También se envía si se recibe un intento de conexión a un **puerto cerrado**, a la cual se debe responder con Flags **R(reset)**, **A (Ack)**.

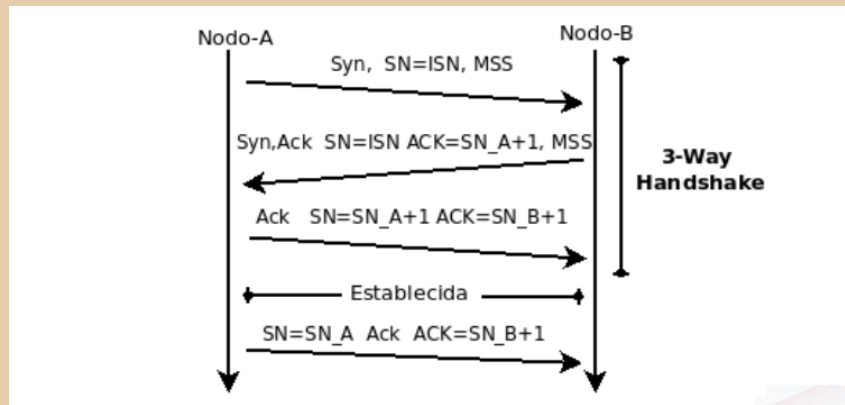


Figura 10: Three-Way Handshake

9.2.2 Fin de Conexión

Se intercambian 3 o 4 segmentos para finalizar la conexión, y cualquier extremo puede iniciar el cierre.

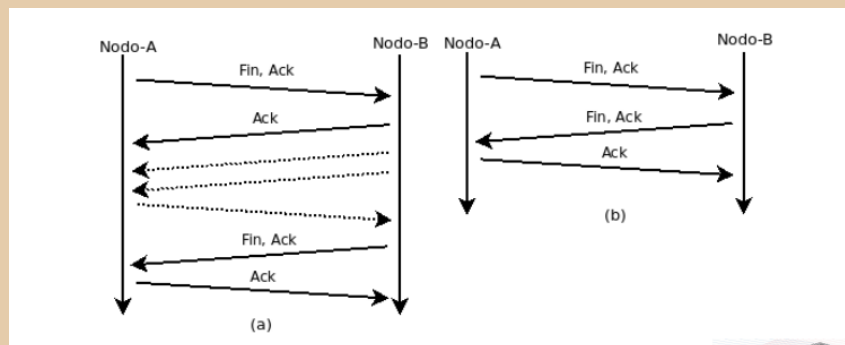


Figura 11: Finalización en TCP

9.2.3 Estados de TCP

Durante la conexión, los nodos transitan distintos estados.

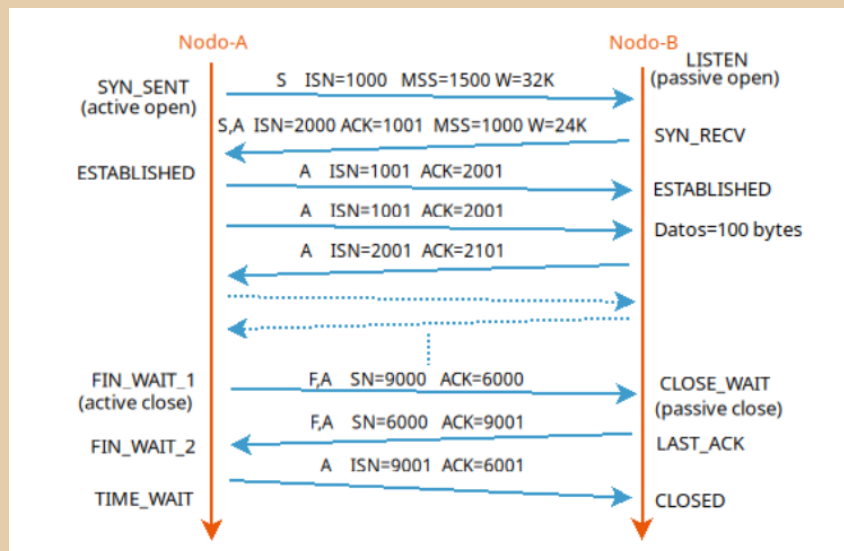


Figura 12: Estados en TCP

Recordar...

Las conexiones TCP empiezan con un **Three-Way Handshake**, con los flags

1. SYN
2. SYN, ACK
3. ACK

El **Flag de Reset (R)** provoca un cierre instantáneo de una conexión, que se puede volver a abrir. Para el **cierre correcto** de una conexión, también hay un intercambio de 3/4 mensajes.

9.3 Intercambio de Datos

Maximum Segment Size (MSS)

Es la máxima longitud de segmento que se puede recibir sin contar la cabecera. Es una opción mandatoria, no es un campo en el header. Se envía en el three-way handshake. Es diferente para **cada sentido de la conexión**.

TCP tiene una **pseudocabecera** que se construye localmente para el cálculo del checksum. Sirve para verificar la integridad del segmento.

Los **sequence numbers** son un número de 32 bits cuyo valor de **ISN** (initial sequence number) se determina localmente en cada extremo. Cada byte enviado sobre una conexión TCP tiene un número de secuencia, y se incrementa según los bytes transmitidos y reconocidos. Indica la ubicación de cada byte en el stream.

Es un canal **Full-Duplex** (bidireccional), donde un extremo puede enviar datos hasta que alcance el tamaño de la ventana de recepción sin esperar un ACK.

El número de ACK indica el siguiente byte que se espera recibir. Si el segmento llega con errores, es retransmitido.



Figura 13: Secuencias en TCP

recordar

Puedo enviar datos hasta llegar a la ventana de transmisión sin tener que esperar un ACK. Si el segmento no tiene errores, me tienen que enviar un ACK. Sino, tengo que retransmitir.

9.4 Control de Flujo

Cada extremo anuncia el tamaño de su **ventana de recepción** al inicio de la sesión, y pueden ser **distintos**.

Se utiliza para que un extremo no envíe datos más rápido de lo que el otro extremo los pueda procesar. Si los datos no son leídos por la aplicación, el extremo debe anunciar su ventana de recepción con un menor tamaño.

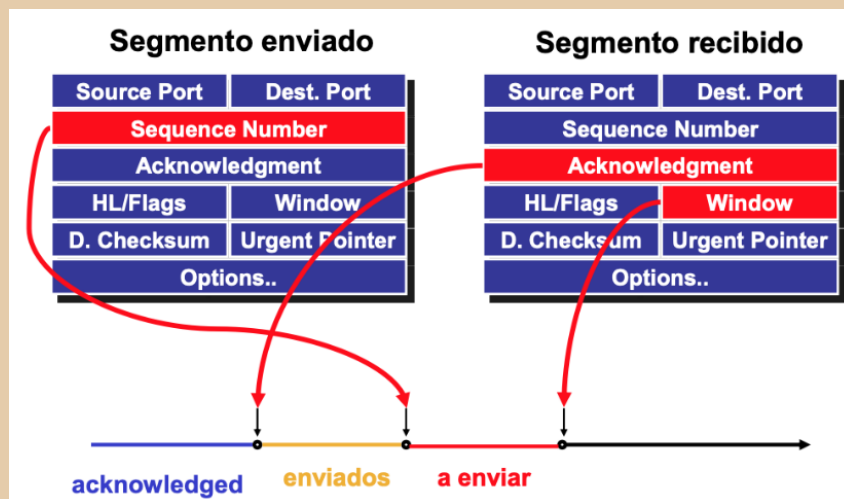


Figura 14: Ventana deslizante en TCP

RTO y RTT

Round Trip Time: es el tiempo entre que se envía un segmento y ACK de reconocimiento. El RTT varía continuamente.

Retransmission TimeOut: el tiempo que debe pasar para retransmitir un paquete.

El **Maximum Segment Size** se comunica en el campo de **opciones**.

9.4.1 Timers

El **Persist Timer** se utiliza cuando la ventana de recepción se reduce a cero. Una vez que se reduce la ventana, el emisor va enviando paquetes de **1 Byte** cada un intervalo de tiempo que va incrementando (**Back-Off**). Esto se hace hasta no recibir un ACK con `wind = 0`.

El **Keepalive Timer** sirve para mantener una conexión abierta cuando no hay intercambio de datos. Se utiliza para interrogar al otro extremo por su estado, y funciona de forma muy similar al *persist timer*.

Recordar...

El **Control de Flujo TCP** se ocupa de evitar el buffer del receptor cuando el emisor es más rápido, y lo hace mediante una **Ventana de Recepción** dinámica (varía a lo largo de la conexión). El tamaño de la ventana se comunica mediante un campo en la **cabecera** llamado **Window Size**.

9.5 Control de Congestión

El control de congestión sirve para evitar degradar la calidad de la red.

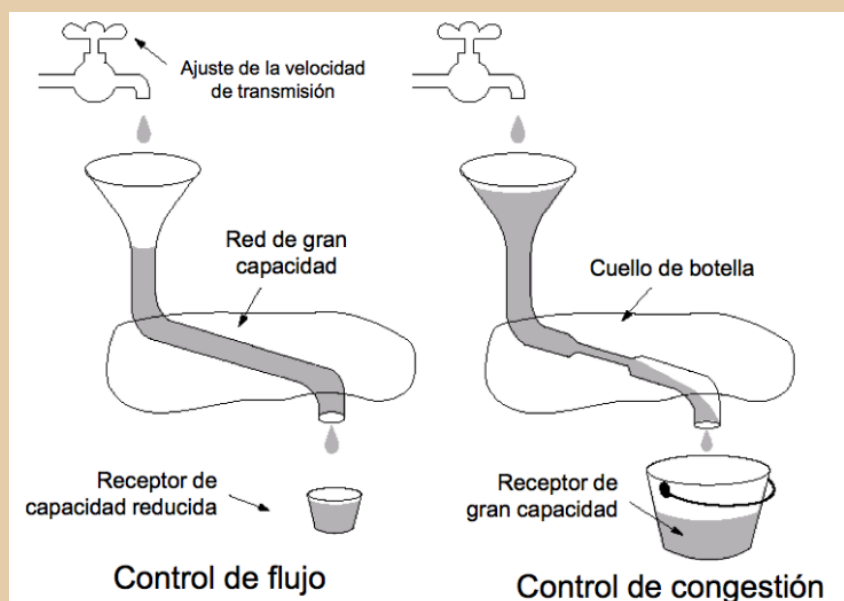


Figura 15: Control de congestión vs control de flujo

9.5.1 Modelo end-to-end

Los extremos son la fuente de la demanda. Ellos deben estimar los **tiempos** y el **grado de congestión**, y en base a ello reducir la demanda.

Los nodos intermedios monitorean el estado de la red.

Utiliza tres variables.

1. **cwnd:** ventana de congestión (congestion window).
2. **rcv_win:** ventana del receptor, publicada en el segmento (receive window).
3. **ssthresh:** umbral de slow start que actualiza cwnd (slow start threshold).

$$\text{ventana} = \min(\text{rcv_wnd}, \text{cwnd})$$

La fase de **Slow Start** se encarga de inicializar el sistema y descubrir la congestión rápidamente, incrementando **cwnd** hasta alcanzar la congestión y estima su valor óptimo mediante pérdida de segmentos.

Luego de cada segmento validado, la $cwnd$ se duplica. Cuando $cwnd \geq ssthresh$, TCP detiene el crecimiento de $cwnd$.

Cuando $cwnd > ssthresh$, por cada ACK $cwnd = cwnd + 1$, es decir, se incrementa de forma lineal.

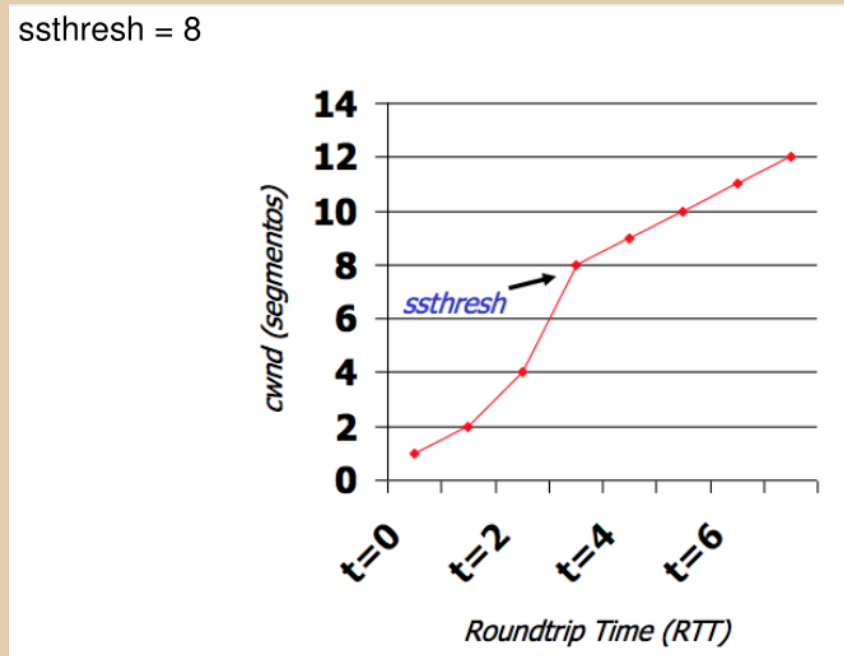


Figura 16: $ssthresh$ y $cwnd$

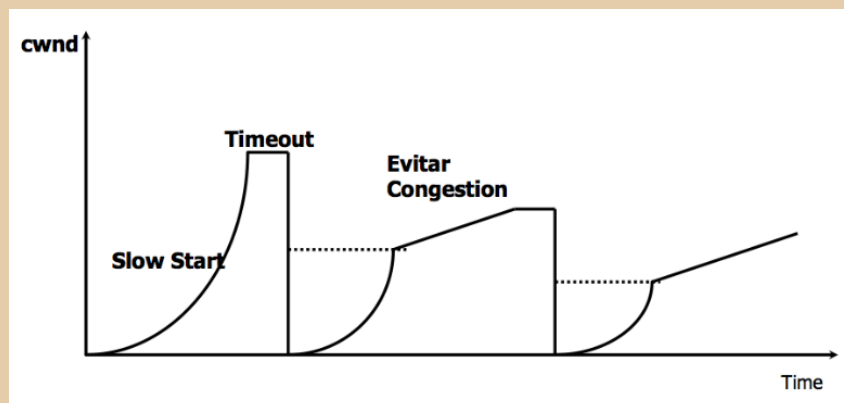


Figura 17: Gráfico completo

La **detección de paquetes perdidos** se hace esperando el RTO , que usualmente es $2 * RTT$.

9.5.2 Fast Retransmit, Fast Recovery

Cuando se recibe un segmento fuera de orden se debe enviar un ACK, lo cual provoca **duplicación de ACKs**.

Fast Retransmit

Al recibir el **tercer ACK repetido** se retransmite sin esperar el RTO .

Luego de esa retransmisión, se pasa a **Fast Recovery** reduciendo $cwnd = ssthresh = cwnd_anterior / 2$. Si hay un **timeout**, la $cwnd$ se reinicia a 1.

10 QUIC

QUIC es un protocolo de capa de transporte que viene a solucionar problemas de TCP:

- **Head of Line Blocking:** la pérdida de un segmento bloquea el envío de los segmentos subsiguientes hasta que sea recuperado.
- **Retardo de Establecimiento de Conexión:** el 3WH tarda mucho.
- **Tamaño de Cabecera Fijo.**
- **Identificación de Conexiones:** con TCP se necesita una IP y puerto TCP para identificar una conexión. Si se cambia la IP, el identificador se pierde.

QUIC

Protocolo de transporte **orientado a conexión**, diseñado para permitir intercambios rápidos con **0-RTT**. Está **encapsulado en UDP**, pero usa conexiones llamadas **streams**.

10.1 Paquetes y Frames

Los extremos se comunican mediante **paquetes**, y a su vez cada paquete QUIC contiene **frames**.

Un datagrama UDP encapsula uno o más paquetes QUIC, un paquete QUIC encapsula uno o más frames de mismo stream o de **diferentes streams**.

Hay distintos tipos de paquetes:

- **Version Negotiation:** solo los envía un servidor cuando no conoce la versión de QUIC del cliente.
- **Initial:** transporta el primer frame Crypto entre cliente y servidor, para ejecutar el cambio de claves.
- **0-RTT:** envía datos de aplicación inmediatamente, antes del handshake. Requiere una conexión previa.
- **Retry:** lo envía el servidor cuando quiere validar la IP del cliente antes del intercambio de claves.
- **1-RTT:** se usa después de la versión y las claves, contienen streams y acks.

10.2 Streams

Streams

Representa un flujo de bytes dentro de una conexión QUIC, y se crean mediante el envío de datos. Pueden ser creados por cualquiera de los extremos, y la cantidad es arbitraria. Los identifica un Stream ID, que no puede ser reusado durante la conexión.

Los límites de un frame tienen que estar dentro de los límites del **control de flujo** indicado por el otro extremo.

Ciclo de Vida y Estados de un Stream en QUIC

En el protocolo QUIC, los *streams* operan como canales lógicos independientes que atraviesan un ciclo de vida estructurado para garantizar la multiplexación eficiente sin sufrir el problema de bloqueo de cabeza de línea (Head-of-Line Blocking). Todo flujo nace en un estado **Idle** (inactivo) hasta que uno de los extremos inicia la comunicación, pasando al estado **Open** (abierto), donde ocurre el intercambio de información (unidireccional o bidireccional). A medida que la transmisión de datos finaliza mediante la señalización de un flag **FIN**, o si la comunicación se interrumpe abruptamente con un **RESET**, el flujo transita por estados intermedios denominados **Half-Closed** (semi-cerrado local o remoto), lo que permite que un extremo termine de enviar datos mientras aún sigue recibiendo los del otro lado. Finalmente, cuando ambos sentidos de la comunicación han concluido, el flujo alcanza el estado **Closed** (cerrado), liberando sus recursos de forma completamente aislada sin afectar la estabilidad ni el rendimiento del resto de los flujos activos en la conexión.

Mediante los *Streams* se logra **multiplexar** intercalando frames de múltiples streams dentro de uno o más paquetes QUIC.

10.3 Conexiones

CID

Cada conexión posee identificadores de conexión (CIDs), elegidos por cada extremo de forma **independiente** donde uno elige el CID del otro. Los paquetes de cabecera larga incluyen *Destination CID* y *Source CID*, mientras que los de cabecera corta incluyen solo el source.

10.3.1 Handshake

Las conexiones son aseguradas con **Transport Layer Security 1.3 (TLS 1.3)**.

1. El cliente le envía un *Client Hello* en un frame *CRYPTO* mediante un paquete *QUIC Initial*.
2. El servidor responde con un **Server Hello** y la información de seguridad (varios paquetes QUIC).
3. El cliente finaliza el handshake.

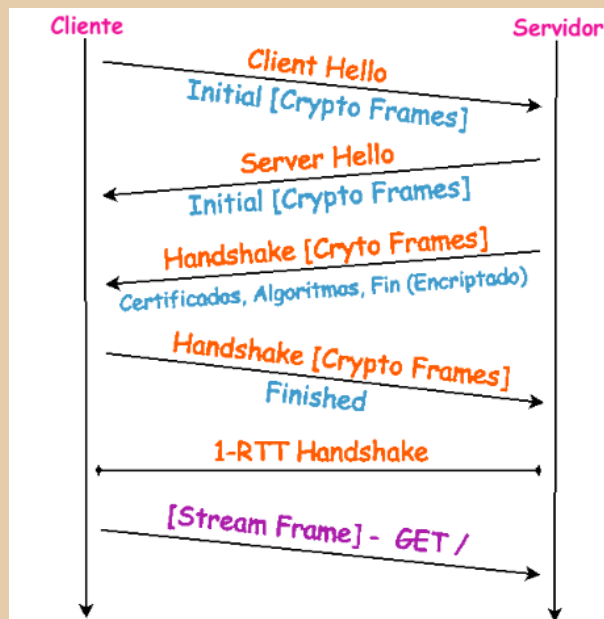


Figura 18: Handshake de QUIC

También se puede hacer **0-RTT Handshake**, que permite enviar datos reutilizando parámetros negociados en una conexión previa que recuerda el cliente.

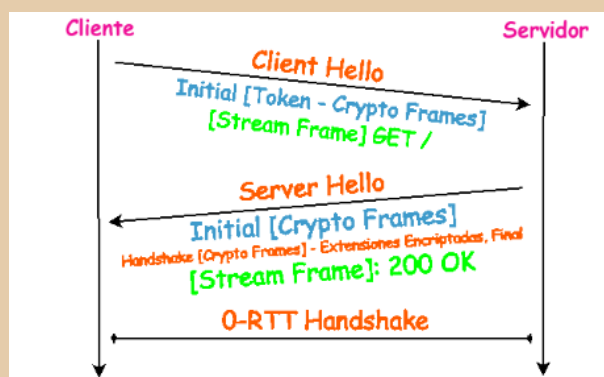


Figura 19: 0-RTT Handshake de QUIC

En QUIC, cada paquete en un datagrama UDP debe ser reconocido individualmente por ACKs. Si se pierde un paquete no se retransmite, sino que se genera otro paquete con un nuevo número de paquete.

10.3.2 Cierre de Conexiones

Las conexiones se pueden cerrar de tres formas distintas:

- **Tiempo de Inactividad:** el cliente y servidor anuncian su tiempo de inactividad, y se elige el menor. Si pasa este tiempo y ninguno envió datos, se cierra la conexión **silenciosamente**.
- **Cierre Inmediato:** ocurre cuando un extremo envía un frame *connection_close*. Todos los streams se cierran en el acto.

- **Reset sin Estado:** se da cuando un extremo recibe paquetes que no puede asociar a una conexión activa.

10.4 Control de Flujo

El receptor necesita limitar la cantidad de datos que puede recibir en su buffer.

Niveles de Control de Flujo QUIC

Nivel de Stream: previene que un stream consuma el buffer completo de la conexión.

Nivel de Conexión: previene que un emisor exceda la capacidad del buffer del receptor.

Los **límites** para todos los streams se setean en el *handshake*, y se pueden **incrementar** a lo largo del tiempo; si se intenta reducir el paquete es válido pero no tiene efecto alguno.

*El receptor cierra una conexión con `flow_control_error` si el emisor viola los límites. En este caso se considera al emisor como **bloqueado**, y el mismo emisor debe enviar frames `DATA_BLOCKED` para indicar que tiene datos a enviar pero está bloqueado por límites de control de flujo.*

Si el emisor envía datos hasta llegar al **límite** entonces no puede enviar más y se lo considera bloqueado.

10.5 Transport Layer Security (TLS) 1.3

Provee un canal seguro entre dos extremos mediante autenticación del servidor al cliente (opcionalmente al revés) además de brindar confidencialidad e integridad de los datos que se transmiten.

El TLS 1.3 se compone de dos factores:

- **Handshake Protocol:** autenticación, negociación de protocolo criptográfico e intercambio de claves.
- **Record Protocol:** transferencia segura de datos utilizando claves establecidas en el handshake.

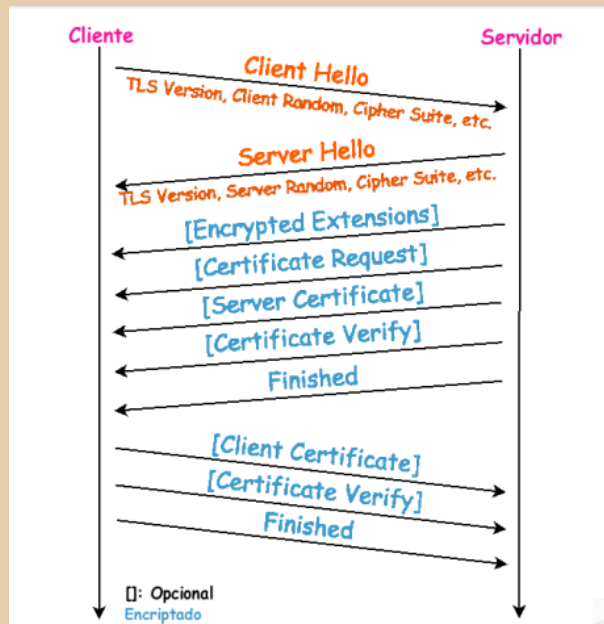


Figura 20: Handshake con TLS 1.3

Los mensajes TLS se transportan en frames *crypto*.

En resumen... QUIC es un protocolo de transporte sobre UDP que permite encapsular varios paquetes dentro de un datagrama UDP, y a su vez varios frames dentro de un paquete QUIC. Tiene menos RTT sin dejar de lado la seguridad. Combina lo mejor de TCP con lo mejor de UDP.

11 Domain Name System

Mapa direcciones (IP/IPv6) a nombres. Los nombres se pueden jerarquizar de dos formas: organizacionalmente y territorialmente. Funciona sobre UDP y utiliza TCP para transferencia de zonas, ambas en puerto 53.

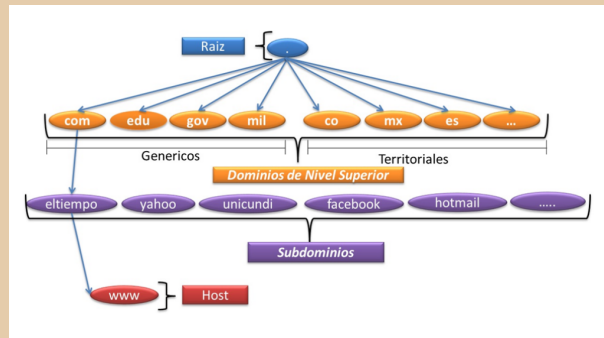


Figura 21: Jerarquía de nombres DNS

Zonas

Llamamos zona a la unidad de administración delegada a un individuo o organización. Es la administración de un determinado espacio de nombres DNS.

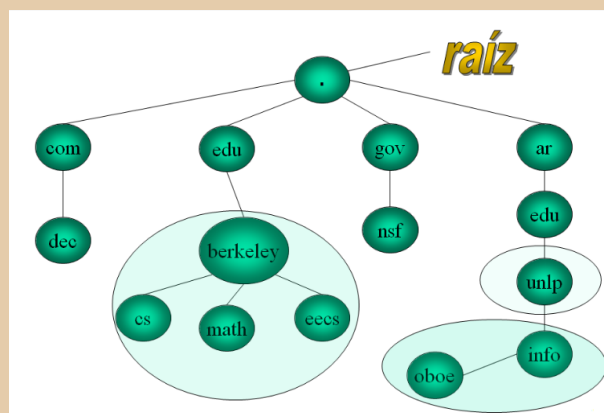


Figura 22: Visualización de zonas DNS

11.1 Registros de Recursos

Son los nombres lógicos con los que se almacenan los datos del servidor.

- **A (Address):** mapea un nombre a una dirección IP.
- **PTR (Pointer):** mapea una dirección IP a un nombre (vice-versa de A).
- **CNAME (Canonical Name):** apunta a otro nombre, se lo conoce como *alias*.
- **MX (Mail Exchanger):** registro que hace funcionar al correo electrónico. Le dice a todo Internet **qué servidor es el encargado de recibir los mails** de ese dominio.

- **NS (Name Server):** indica cuáles son los servidores DNS que tienen la información oficial y actualizada.
- **SOA (Start Of Authority):** para pedir parámetros del dominio.

11.2 Servidores DNS

El **servidor raíz** delega a todos TLDs, y no permite consultas recursivas. Proporcionan referencias directas a servidores de dominio de segundo nivel (com, edu, gov, ...). Los **servidores autoritativos** se encargan de una zona o sub-dominio de nombres, y puede subdelegar.

Los **servidores locales** son los que se consultan dentro de una red. Mantienen caché, permiten consultas recursivas a los hosts internos y pueden ser autoritativos.

Luego están los **Open Name Servers**, que son servidores de DNS que funcionan como si fueran locales para cualquier cliente. Los **Forwarded Name Servers** funcionan como proxies de otros DNSs internos, e interactúan directamente con el sistema DNS exterior.

Los **Primary Name Servers** almacenan la información de su zona en una database local, y son responsables de mantener la información actualizada. Los **Secondary Name Servers** obtienen los datos desde otro servidor que tenga la autoridad para esa zona, y copia la información. Este proceso se llama *transferencia de zona* y en vez de hacerse por UDP se hace por TCP.

11.3 Proceso de Consulta

Mecanismo de Resolución de Nombres (DNS)

Todo host cuenta con la configuración de un **servidor DNS local** (resolver). Cuando un dispositivo necesita resolver un nombre de dominio, envía su petición inicial a este servidor local, a partir de lo cual pueden darse dos escenarios:

- **Respuesta Autoritativa (Fidedigna):** Si el servidor local es la autoridad para el dominio consultado, significa que aloja exactamente esa porción de la base de datos distribuida. En este caso, posee el mapeo directo y le devuelve la respuesta al host de forma inmediata.
- **Búsqueda Externa (Delegación):** Si el servidor local no es fidedigno para dicho dominio, deberá iniciar una búsqueda externa consultando a un **Root Server** (Servidor Raíz).

Dinámica de Peticiones (Recursivas vs. Iterativas):

Para evitar la sobrecarga y el colapso de la infraestructura global, la petición dirigida al Root Server es estrictamente **iterativa**. El Root Server no busca la IP final, sino que se limita a devolver la dirección de un servidor intermedio (como un servidor TLD).

A partir de allí, el servidor local envía una petición **recursiva** a este servidor intermedio. Al ser recursiva, le delega la responsabilidad de continuar interrogando a la jerarquía de servidores hasta alcanzar al servidor fidedigno definitivo. Una vez hallado el mapeo correcto, la respuesta viaja de regreso hasta llegar al host original.

Se utiliza **Caching** para reducir el tráfico en búsquedas de dominios no locales, donde las entradas a esta memoria se temporizan.

11.4 Flags DNS

Cuando realizamos una **consulta DNS**, la misma puede retornar distintos flags.

- **qr:** query.
- **rd:** recursion desired, el cliente desea que la consulta se resuelva recursivamente.
- **ra:** recursion available, el servidor tiene la recursión habilitada.

El **TTL** (Time to Live) indica el tiempo que se mantiene la respuesta en caché (en segundos).

12 File Transfer Protocol - FTP

FTP

Protocolo utilizado para **compartir y acceder a archivos**. Se encapsula en TCP en puerto 20 y 21 para las conexiones de control y transferencia.

El cliente utiliza un puerto TCP aleatorio, mientras que el servidor utiliza el **21 para el control** y el **20 para transferencia**.

13 Correo Electrónico

Hay tres protocolos principales para el correo electrónico, con distintos fines:

- **Distribución:** se utiliza el protocolo SMTP o Extended SMTP (ESMTP).
- **Entrega Final:** permite al usuario gestionar su correo a través de una máquina remota. Son POP3, IMAP o HTTP.

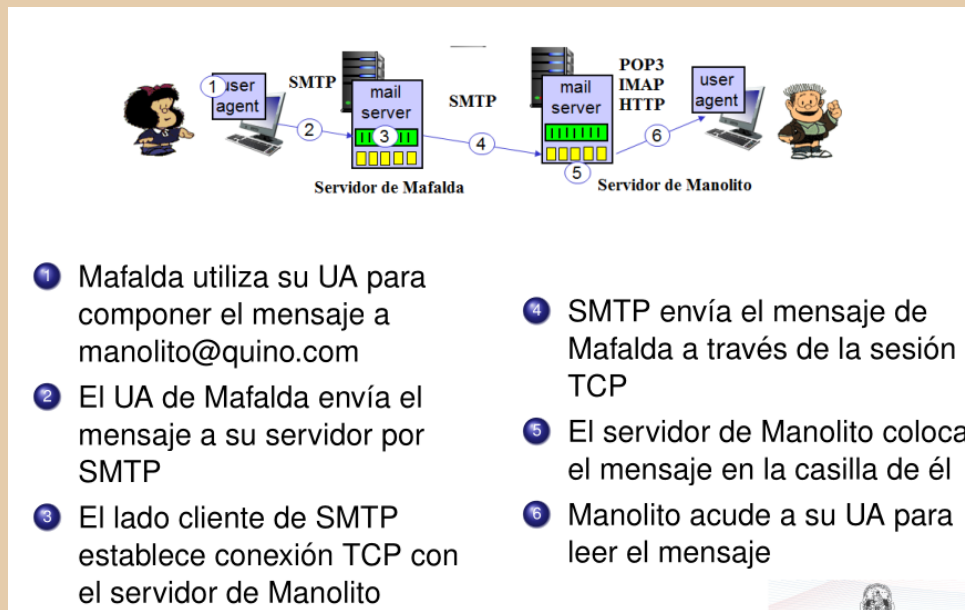


Figura 23: Ciclo de un Correo Electrónico

13.1 Simple Mail Transfer Protocol - SMTP

Definición

Tiene el objetivo de transferir mails de manera confiable y segura. El usuario coloca una copia del mensaje en el *spool* y el cliente en *background* envía los mensajes. El cliente barre los mensajes cada un cierto tiempo.

Se utiliza DNS para resolver el nombre del destino en una dirección IP y luego se establece la conexión TCP con el servidor.

1. El sender abre una conexión TCP al **puerto 25 del receptor**.
2. El **User Agent** consulta por el registro A de su servidor DNS.
3. El **SMTP Server** hace una consulta DNS para obtener los registros **MX** del dominio destino.
4. Hacen un handshake entre el receptor y el emisor.

13.1.1 Multipurpose Internet Mail Extensions - MIME

MIME

Es una extensión para enviar archivos ejecutables y multimedia mediante mail con ASCII de 7 bits, que era lo que permitía SMTP.

13.2 Post Office Protocol 3 - POP3

POP3

Tiene como objetivo obtener el correo electrónico del buzón remoto y almacenarlo localmente en la PC del usuario para su lectura. Utiliza el **TCP en puerto 110**.

Puede ser *download and delete* o *download and keep*. Según la modalidad borra o no los mensajes en la fase de actualización. En caso de que esté en *download and delete*, no permite sincronismo entre otros dispositivos.

13.3 IMAP

IMAP

Asocia cada mensaje a una carpeta, sobre las cuales se pueden desplazar los mails. A diferencia de POP3, permite sincronismo entre otros dispositivos, ya que se accede al servidor para leer y no se borran los leídos.

Funciona sobre **TCP en puerto 143**.

También incorpora descargas parciales de los mensajes.

14 Hypertext Transfer Protocol - HTTP/1.1

Es el protocolo encargado de transferir **recursos** en internet. Los recursos son identificados por una **URI** (Uniform Resource Location). Las URIs tienen dos formatos: URN (Uniform Resource Name) o URL (Uniform Resource Location).

Funcionamiento de HTTP

Es un protocolo de solicitud/respuesta **sin estado**, de **texto plano**. Utiliza MIME para enviar datos binarios. Corre sobre **TCP puerto 80**. También existe una versión segura **HTTPS** que usa TLS; corre en **TCP puerto 443**.

Cuando se solicita un recurso tenemos que esperar la respuesta, no hay un identificador que permita relacionarlas. Permite *pipelining*.

14.1 Mensajes y Métodos

1. Los clientes envían **request methods** al servidor mediante HTTP. Este request method indica al servidor qué acción ejecutar.
2. El servidor ejecuta el method.
3. El servidor responde con uno o más **response methods** que incluyen un código de estado, indicando si la solicitud fué exitosa o no, o si debe realizar alguna acción.

Los mensajes tienen la misma estructura: línea de inicio, cabecera y cuerpo (opcional).

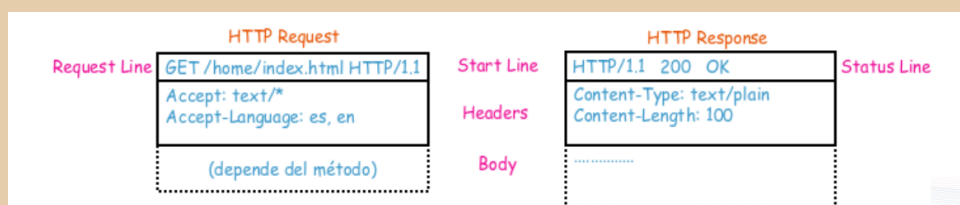


Figura 24: Mensajes HTTP/1.1

La **línea de inicio** es la primera de todos los mensajes HTTP, luego siguen las **cabeceras** de nombre-valor separadas por :, y por último el **body** que depende del método usado y del resultado de la operación.

14.1.1 Algunos Métodos

Los métodos más utilizados son:

- **GET**: el cliente solicita un recurso al servidor. Podrían enviarse datos en el body, pero no es usual. Ejemplo: `http://www.example.com/index.php?pais=Argentina,deporte=tenis`. Los parámetros van después del ?.
- **HEAD**: igual al GET pero solo se obtienen los **metadatos** del recurso (headers). No se envía el recurso solicitado.

- **POST:** envía datos de entrada al servidor, que se mandan en el body. Solicita una respuesta del servidor.
- **PUT:** envía contenido que será almacenado en el webserver.
- **DELETE:** indica contenido a ser eliminado del webserver.

14.2 Funcionamiento

Cada mensaje puede tener ninguna o muchas cabeceras que agregan información adicional a los mensajes. Algunas cabeceras son:

- **Host:** obligatoria, indica el servidor y puerto destino.
- **Date:** día y hora de creación del mensaje.
- **Content-Length:** longitud del cuerpo del mensaje.
- **Content-Type:** tipo de objeto que se envía.

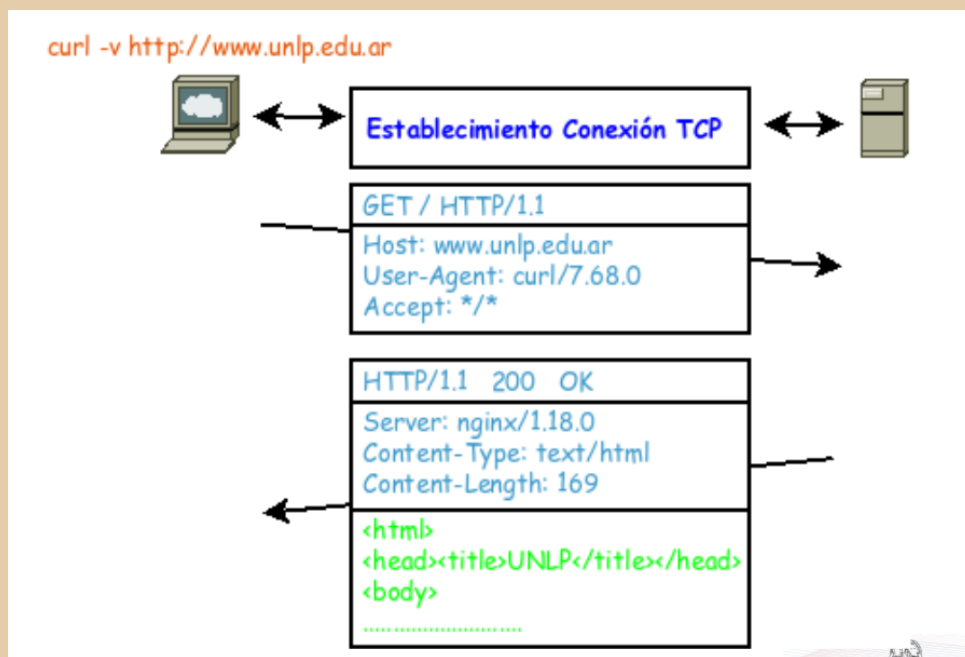


Figura 25: Funcionamiento de HTTP/1.1

Recordar... En **HTTP/1.0** las conexiones eran **no persistentes**, es decir, se transfiere un objeto por cada sesión y por cada recurso se debe abrir y cerrar una sesión TCP. El servidor responde con un objeto y cierra la sesión (algunas implementaciones de HTTP/1.0 soportan conexiones keep-alive, pero no es lo default).

En HTTP/1.1 las conexiones son **persistentes por defecto**, y con las cabeceras *Connection: close* indicamos que queremos cerrar la conexión después de la respuesta del server.

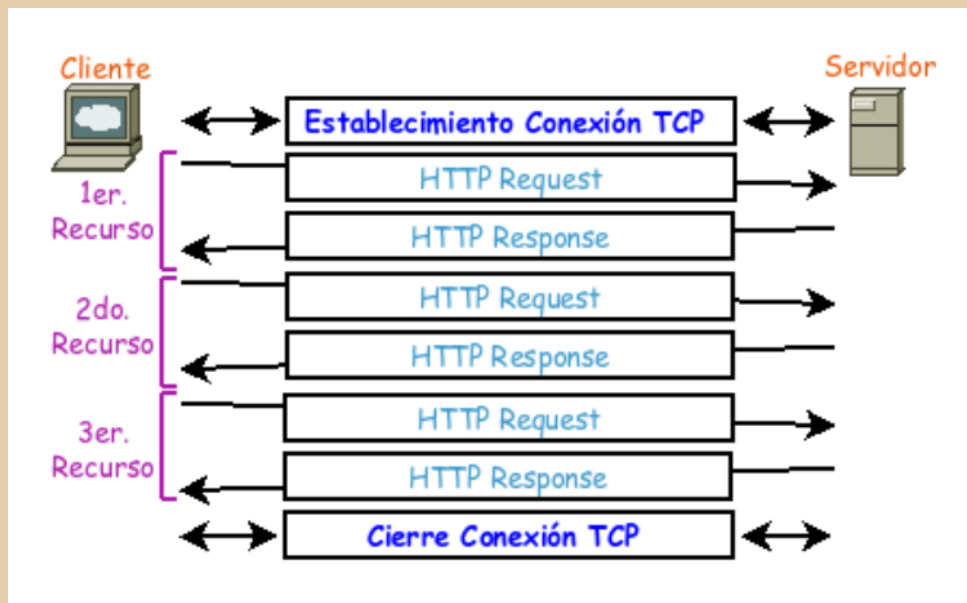


Figura 26: Conexiones persistentes de HTTP/1.1

Concurrencia en HTTP/1.1

Este protocolo permite **pipelining**: el cliente puede enviar muchas solicitudes sin esperar cada respuesta, y el servidor las procesa en paralelo. Tiene que enviar las respuestas en el **mismo orden** que recibió las solicitudes.

Los clientes pueden abrir **conexiones simultáneas** a un servidor, usadas para evitar el *head-of-line blocking*.

Se puede hacer un **GET Condicional** con la cabecera `If_Modified_Since`, donde el servidor retorna el recurso si fué modificado en una fecha posterior a la indicada en el header. Si no fue modificada, se responde con código 304 y el recurso se recupera de caché.

Si el recurso fué designado a una nueva URL, el server responde con 301 (`Moved Permanently`) o 302 (`Found`), y agrega una cabecera `Location` con la nueva URL.

14.3 HTTPS

Es HTTP con TLS, sobre TCP en puerto 443. Los mensajes HTTP se **encriptan y autentican de forma completa**.

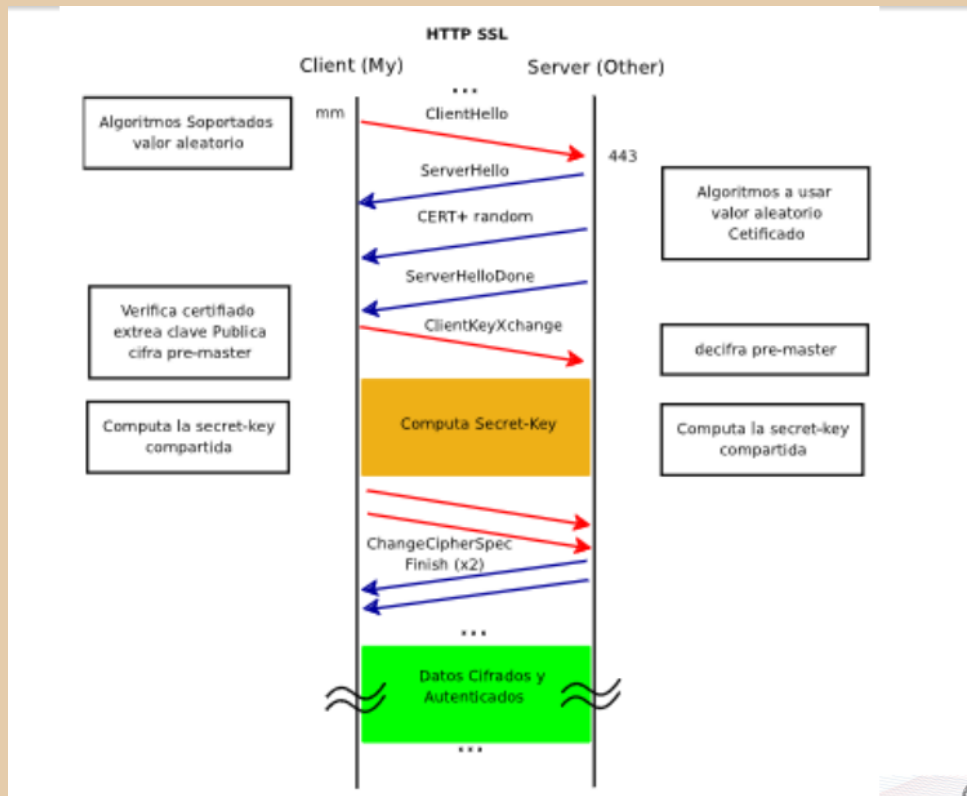


Figura 27: TLS en HTTP/1.1 (HTTPS)

14.4 Cookies

Las cookies permiten a las aplicaciones web del server **manejar estados** conservando información entre peticiones del cliente, ya que HTTP/1.1 no maneja estados por defecto.

Los servers envían información de estado con un header `Set-Cookie`, y en las siguientes solicitudes el cliente retorna una cabecera `Cookie` con los datos recibidos.

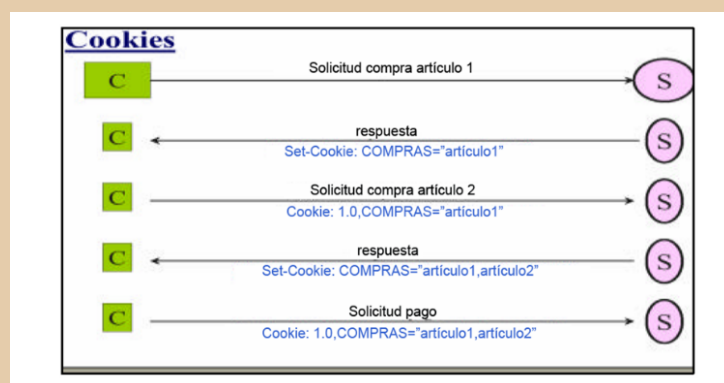


Figura 28: Ejemplo de uso de Cookies

14.5 Caching

Es una característica opcional de HTTP/1.1 que consiste en almacenar de forma local los mensajes de respuesta para reducir el tiempo de respuesta y carga de la red.

Hay que asegurar que los contenidos guardados en caché sean equivalentes a los almacenados en el servidor, generalmente almacenan respuestas exitosas de una solicitud de recuperación.

Si la solicitud es servida desde cache se conoce como **cache hit**, y si no se encuentra en la cache es un **cache miss** y se pide el recurso al webserver. Cuando el recurso está en la caché pero no es válido, se tiene que revalidar utilizando un GET condicional.

15 HTTP/2

HTTP/2 surge para solucionar problemas de HTTP/1.1. Entre ellos, tenemos los siguientes.

- Protocolo sincrónico
- Tiene head-of-line blocking
- Requiere múltiples conexiones a un servidor para utilizar paralelismo

Descripción General

Protocolo de capa de aplicación orientado a conexión, se ejecuta sobre TCP y sus conexiones son persistentes. El cliente no debería abrir más de una conexión TCP al servidor.

Se introducen nuevos conceptos:

1. **Stream:** flujo bidireccional de frames dentro de **una conexión**.
2. **Frame:** unidad básica del protocolo, cada tipo de frame sirve un propósito diferente y un frame pertenece a un stream en particular.

A diferencia de HTTP/1.1 que era de texto plano HTTP/2 es **binario**, lo que lo hace incompatible con otras versiones de HTTP.

Utiliza los mismos puertos que HTTP/1.1 (puerto 80 y 443 para HTTPS).

Para utilizar HTTP/2, es necesario verificar si el otro extremo también soporta este protocolo.

15.1 Frames

Son la unidad más pequeña que transporta todos los datos (headers, payload, etc.). Además de tener los campos de una cabecera tradicional (length, type, flags) también agrega el **Stream Identifier** para indicar a qué stream pertenece el frame.

Hay distintos tipos de frames.

- **HEADERS:** usado para abrir un stream, transporta cabeceras del requerimiento y/o respuesta.
- **DATA:** transporta bytes de longitud variable asociados a un stream.
- **RST_STREAM:** cierra el stream inmediatamente. El emisor no envía nuevos frames, pero si puede recibir.
- **SETTINGS:** transporta parámetros de configuración que afectan a la conexión. Se envían solo en el primer stream, y debe ser enviado por cada extremo.
- **PUSH_PROMISE:** solo los envía el servidor, sirven para avisar de streams que el emisor va a utilizar.
- **PING:** mide el RTT y determina si la conexión aún está activa.
- **GOAWAY:** inicia el cierre elegante de una conexión o indica conexiones de error.
- **WINDOWS_UPDATE:** se utiliza para controlar el flujo en los streams o conexión.

15.2 Streams

Definición

Secuencia bidireccional de frames intercambiados entre cliente y servidor de una conexión. Pueden haber varios abiertos concurrentemente, cada uno identificado con un entero asignado por el extremo que lo inicia. Si lo hace el servidor, es par. Sino es impar. Importa el orden en el que se envían los frames, ya que se procesan en ese orden.

El stream con **identificador 0** se utiliza para enviar mensajes de **control de conexión**. Cada nuevo identificador debe ser mayor a todos los elegidos previamente por ese extremo.

Si un cliente envía una solicitud HTTP en un nuevo stream, el servidor envía la respuesta en ese mismo stream de la solicitud. Cada una de ellas puede consistir de uno o más frames.

Una **solicitud o respuesta HTTP/2** consta de un frame **HEADER** y cero o más frames **DATA**.

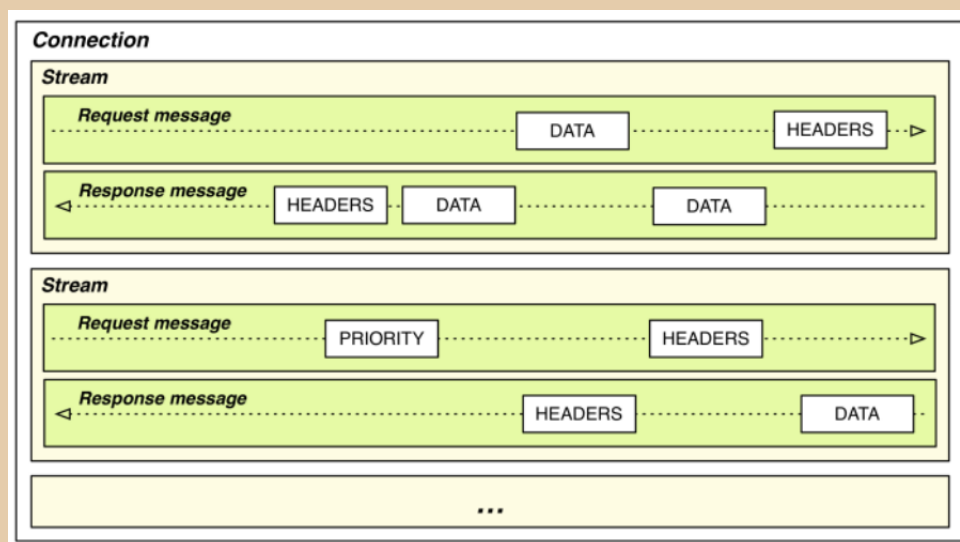


Figura 29: Conexiones en HTTP/2

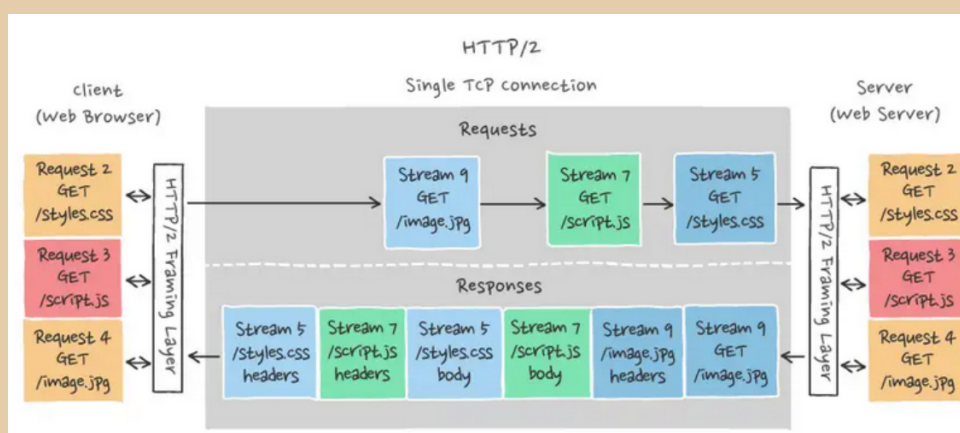


Figura 30: Conexiones en HTTP/2 - 2

15.3 Control de Flujo

Los streams **compiten entre sí** por el uso de la conexión TCP. Como TCP no puede aplicar control de flujo sobre los streams individuales, cada emisor debe mantener una **ventana por cada stream**.

Capacidad de Buffering

Valor entero que indica cuantos bytes de datos puede transmitir el emisor. Es unidireccional: cada extremo anuncia su tamaño máximo de ventana para la conexión y cada stream.

El **frame** SETTINGS permite modificar el tamaño de la ventana al **inicio** de la conexión (el tamaño default es 65.535 bytes), y el límite sólo se aplica sobre los frames DATA.

Recordar... El *WINDOW_UPDATE* puede ser específico para un stream o para la conexión entera, ya que se mantiene una **ventana por cada stream** y para la **conexión como un todo**.

15.4 Server Push

Consiste en que el servidor puede enviar un recurso antes de que el cliente se lo solicite, mandando un **frame** PUSH_PROMISE. Dentro de este push promise se indica cual es el identificador de stream que se usará para la respuesta.

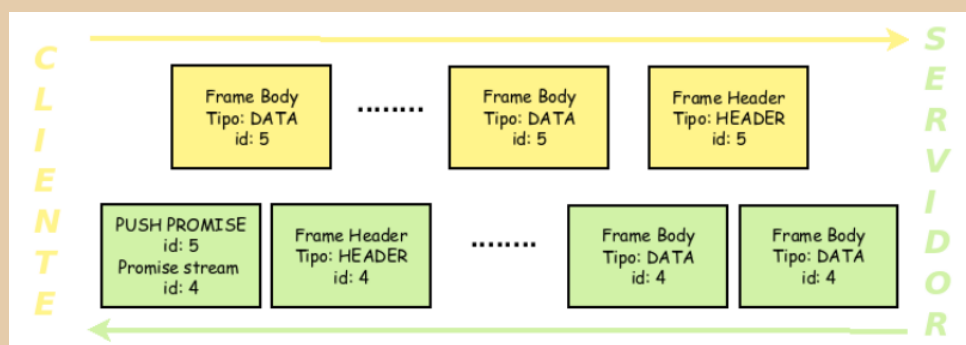


Figura 31: Server push

15.5 Head Compression

En HTTP/1.1 podíamos comprimir solo el payload. En HTTP/2 podemos **comprimir la cabecera y el payload** utilizando **HPACK**, que es un algoritmo de compresión diseñado para comprimir cabeceras. Consiste en una tabla estática predefinida y una tabla dinámica que permite agregar cabeceras a continuación de la estática.

<i>Index</i>	<i>Header Name</i>	<i>Header Value</i>
<i>1</i>	<i>:authority</i>	
<i>2</i>	<i>:method</i>	<i>GET</i>
<i>3</i>	<i>:method</i>	<i>POST</i>
<i>4</i>	<i>:path</i>	<i>/</i>
<i>5</i>	<i>:path</i>	<i>/index.html</i>
<i>6</i>	<i>:scheme</i>	<i>http</i>
<i>7</i>	<i>:scheme</i>	<i>https</i>
<i>8</i>	<i>:status</i>	<i>200</i>
<i>.....</i>		
<i>61</i>	<i>www-authenticate</i>	

Figura 32: HPACK